МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
‹‹СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ››
Институт перспективной инженерии

Дисциплина «Введение в программирование и алгоритмы»

Отчет по лабораторной работе №6
«Одномерные массивы»

Выполнил студент группы
ПИЖ-б-о-26-1(2)
Киркачев М.Е.
Проверил:
Преподаватель по дисциплине
Николаев Е. И.

г. Ставрополь, 2026 г.

Цель работы: научиться работать с одномерными статическими массивами: объявлять, инициализировать, заполнять с клавиатуры и случайными числами. Освоить базовые алгоритмы обработки массивов: поиск суммы, среднего арифметического, минимального и максимального элементов, подсчёт элементов, удовлетворяющих условию. Получить навыки работы с генератором случайных чисел и форматированного вывода массивов.

ХОД РАБОТЫ

https://github.com/Max2311008/my-projects/tree/main/work%20in%20C++/lub_6

ВАРИАНТ №9

Задание 1 (базовый уровень)

Код:

```cpp
#include <iostream>  // Подключаем заголовочный файл
#include <Windows.h>
int main() { // объявляем главную функцию
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    // 3,6,7 строки обрабатывают кириллицу
    int nums[10]; // Создание массива с хранением 10 значений
    for (int i = 0; i < 10;i++) {
        std::cout << i << "  элемент массива -- ";
        std::cin >> nums[i];
    } // Ввод элементов массива с клавиатуры
    int summa = 0;
    for (int i = 0; i < 10;i+=2) {
        summa += nums[i];
    } // Условие вывода суммы массивов с чётными индексами
    std::cout << "\n$============================$\n";
    std::cout << "Сумма массивов с чётными индексами = " << summa << std::endl; // Вывод суммы на экран
    std::cout << "$============================$\n";

    return 0; // Вывод результата
}
```

Результат

```
0  элемент массива -- 74
1  элемент массива -- 24
2  элемент массива -- 13
3  элемент массива -- 674
4  элемент массива -- 257
5  элемент массива -- 224
6  элемент массива -- 463
7  элемент массива -- 62
8  элемент массива -- 356
9  элемент массива -- 24

$=======================================$
Сумма массивов с чётными индексами = 1163
$=======================================$
```

Задание 2 (средний уровень)

Код:

```cpp
#include <iostream>  // Подключаем заголовочный файл
#include <Windows.h>
#include <ctime>
int main() { // объявляем главную функцию
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    // 3,6,7 строки обрабатывают кириллицу
    int nums[15];
    std::srand(static_cast<unsigned>(std::time(nullptr)));
    for (int i = 0; i < 15; i++) {
        nums[i] = std::rand() % 51;
        std::cout << nums[i] << " ";
    }
    int first = -1;
    int last = -1;
    for (int i = 0; i < 15;i++) {
        if (nums[i] == 0) {
            if (first == -1) {
                first = i;
            }
            last = i;
        }
    }

    if (first == -1 || first == last) {
        std::cout << "\nВ массиве меньше двух нулей. Сумма = 0\n";
    }
    else {
        int sum = 0;
        for (int i = first + 1; i < last; i++) {
            sum += nums[i]; // Складываем элементы строго между нулями
        }
        std::cout << "\nПервый ноль на позиции: " << first << "\n";
        std::cout << "\nПоследний ноль на позиции: " << last << "\n";
        std::cout << "\nСумма элементов между ними = " << sum << "\n";
    }

    return 0;
}
```

Результат

```
50 37 23 0 1 17 7 12 29 14 12 26 0 27 46
Первый ноль на позиции: 3

Последний ноль на позиции: 12

Сумма элементов между ними = 118
```

Задание 3 (повышенный уровень)

Код:

```
if (number == 1) {
    std::srand(static_cast<unsigned>(std::time(nullptr)));
    for (int i = 0; i < SIZE; i++) {
        arr[i] = 1 + std::rand() % 100;
    }
}
else {
    std::cout << "Введите 15 чисел (от 1 до 100):\n";
    for (int i = 0; i < SIZE; i++) {
        int tmp;
        while (true) {
            std::cout << "Элемент [" << i << "]: ";
            std::cin >> tmp;
            if (std::cin.fail() || tmp < 1 || tmp > 100) {
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                std::cout << "ОШИБКА! Введите число от 1 до 100: ";
            }
            else {
                arr[i] = tmp;
                break;
            }
        }
    }
}
printArray(arr, SIZE, "Исходный массив");

// Создаем перевернутый массив
int reversed[SIZE];
for (int i = 0; i < SIZE; i++) {
    reversed[i] = arr[SIZE - 1 - i];
}
printArray(reversed, SIZE, "Реверс массива");

// Проверка на палиндром (независимо от способа ввода)
bool Palindrom = true;
for (int i = 0; i < SIZE / 2; i++) { // Идем только до середины
    if (reversed[i] != arr[i]) {     // Если нашли отличие
        Palindrom = false;
        break;
    }
}
```

Результат

```
ВАШ ВЫБОР: 2
Введите 20 чисел (от 1 до 100):
Элемент [0]: 1
Элемент [1]: 2
Элемент [2]: 3
Элемент [3]: 4
Элемент [4]: 5
Элемент [5]: 6
Элемент [6]: 7
Элемент [7]: 8
Элемент [8]: 9
Элемент [9]: 10
Элемент [10]: 10
Элемент [11]: 9
Элемент [12]: 8
Элемент [13]: 7
Элемент [14]: 6
Элемент [15]: 5
Элемент [16]: 4
Элемент [17]: 3
Элемент [18]: 2
Элемент [19]: 1
Исходный массив:    1   2   3   4   5   6   7   8   9  10  10   9   8   7   6   5   4   3   2   1
Реверс массива:    1   2   3   4   5   6   7   8   9  10  10   9   8   7   6   5   4   3   2   1

$==== ВЫВОД РЕЗУЛЬТАТА ====$
Массив является Палиндромом
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Раздел А. Основы массивов:

1. Массив — это структура данных, представляющая собой набор элементов одного типа, расположенных в памяти последовательно (непрерывным блоком). Один тип элементов, фиксированный размер, последовательное расположение, доступ по индексу, время доступа.
2. Int arr[20]
3. Int arr[3] = {1, 2, 3};
   Int arr[3] = {1, 2};
   Int arr[] = {1, 2, 3};
4. Размер массива должен быть известен на этапе компиляции
5. Через [] (arr[0]). При выходе за границы будет аварийное завершение

Раздел Б. Заполнение и вывод:

1. For (int I = 0; I < SIZE; i++){
   Std::cin >> arr[i];}
2. Библиотеки #include <cstdlib>; #include <ctime>
   Srand(time(nullptr)); arr[i] = rand() % 100;
3. Min + rand() % (max – min + 1);
4. В строку → cout << arr[i] << «…»;
   В столбец → cout << arr[i] << «…» << endl(\n);
5. Задаёт стартовое число генератору, чтобы при каждом запуске числа были разными

Раздел В. Базовые алгоритмы:

1. Int sum = 0;
   For (I = 0; I < SIZE; i++);{
   Sum += arr[i]}

2. Double avg = static_cast<double>(sum) / SIZE; Чтобы избежать целочисленного деления
3. Начать с min = arr[0], потом сравнить:
   if (arr[i] < min) {
   min = arr[i];}
4. Задать счётчик count = 0 и при условии добавлять 1(count++)
5. Int first = -1; в цикле сравнить этот элемент и сохранить индекс(first = i) и сделать break

Раздел Г. Продвинутые алгоритмы:

1.

```
for (int i = 0; i < SIZE / 2; i++) {
    int temp = arr[i];
    arr[i] = arr[SIZE - 1 - i];
    arr[SIZE - 1 - i] = temp;
}
```

2. ВПРАВО:

```
int shiftArr[SIZE];
for (int i = 0; i < SIZE; i++) {
    shiftArr[i] = arr[i];
}

int last = shiftArr[SIZE - 1];
for (int i = SIZE - 1; i > 0; i--) {
    shiftArr[i] = shiftArr[i - 1];
}
shiftArr[0] = last;
printArray(shiftArr, SIZE, "Сдвиг вправо на 1");
```

ВЛЕВО:

```
int shiftLeft[SIZE];
for (int i = 0; i < SIZE; i++) {
    shiftLeft[i] = arr[i];
}

int first = shiftLeft[0];
for (int i = 0; i < SIZE - 1; i++) {
    shiftLeft[i] = shiftLeft[i + 1];
}
shiftLeft[SIZE - 1] = first;
```

```
printArray(shiftLeft, SIZE, "Сдвиг влево на 1");

return 0;
}
```

3. Нужно сдвинуть все элементы, стоящие после него, на одну позицию влево. Значение arr[k] перезапишется как arr[k+1]. Размер массива уменьшится на 1

4. Нужно сдвинуть все элементы начиная с k на одну позицию вправо, сдвиг нужно делать с конца чтобы не потерять данные, а размер массива увеличится на 1
5. Алгоритм сортировки, при котором массив проходится несколько раз. В каждом проходе попарно сравниваются соседние элементы, если они стоят в неправильном порядке, то они поменяются местами.

Раздел Д. Анализ кода и ошибки:

1. 1 2 3 0 0
2. Conts in n
3. Будет ошибка, выход за границу массива
4. Double avg = static_cast<double>(sum) / SIZE
   У std::array есть метод .size(), обычный массив при передаче в функцию теряет размер и std::array можно присвоить целиком,а обычный массив – только поэлементно в цикле.

Вывод: благодаря этой лабораторной работе я смог научиться работать с одномерными статическими массивами: объявлять, инициализировать, заполнять с клавиатуры и случайными числами. Освоил базовые алгоритмы обработки массивов: поиск суммы, среднего арифметического, минимального и максимального элементов, подсчёт элементов, удовлетворяющих условию. Получил навыки работы с генератором случайных чисел и форматированного вывода массивов.